

FICHE DE PROCÉDURE

CinéBase — Banque de films en ligne

SAÉ 2.03 Mettre en place une solution informatique pour l'entreprise

Projet SAÉ RT

IUT Réseaux & Télécommunications

Année 2025–2026

Technologies : Python · Django · MariaDB ·

Nginx

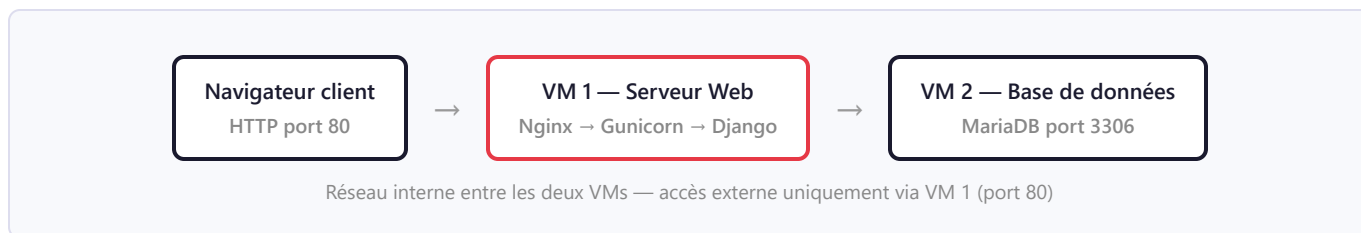
1. PRÉSENTATION DU PROJET

CinéBase est une application web de gestion collaborative de films, développée en Python/Django dans le cadre de la SAÉ RT. Elle permet à des utilisateurs inscrits de consulter un catalogue de films, de lire et déposer des commentaires notés, et aux administrateurs de gérer l'ensemble du contenu (films, acteurs, catégories, utilisateurs). L'application distingue deux profils d'utilisateurs : *amateur* et *professionnel*, afin de calculer des moyennes de notes différenciées.

Le déploiement final repose sur une architecture **2 machines virtuelles** : un serveur web (Django + Gunicorn + Nginx) et un serveur de base de données (MariaDB), communiquant sur un réseau interne.

2. INFRASTRUCTURE MATÉRIELLE ET LOGICIELLE

ARCHITECTURE RÉSEAU



DÉTAIL DES MACHINES VIRTUELLES

VM 1 — Serveur Web IP : [IP_VM_DJANGO]	VM 2 — Base de données IP : [IP_VM_SQL]
OS : Debian / Ubuntu Linux	OS : Debian / Ubuntu Linux
Nginx : Reverse proxy HTTP (port 80)	MariaDB : Serveur SQL (port 3306)
Gunicorn : Serveur WSGI Python (port 8000 local)	Base : cinebase_db (utf8mb4)
Python 3 : Environnement virtuel (venv)	Utilisateur SQL : cinebase_user
Django ≥ 4.2 : Framework web applicatif	Accès réseau : bind-address = 0.0.0.0
mysqlclient : Connecteur Python → MariaDB	Droits : GRANT ALL sur cinebase_db depuis IP VM 1
Pillow : Gestion des images (affiches, photos)	
Git : Clonage du dépôt GitHub	

DÉPENDANCES LOGICIELLES (REQUIREMENTS)

Bibliothèque	Rôle
django ≥ 4.2	Framework MVC — routage, ORM, templates, authentification
gunicorn	Serveur WSGI de production pour Django

mysqlclient	Pilote Python pour connexion à MariaDB/MySQL
pillow	Traitement des images uploadées (affiches de films, photos d'acteurs)

3. FONCTIONNEMENT DE L'APPLICATION

MODÈLE DE DONNÉES

Table	Champs principaux	Relations
Personne	username, email, mot_de_passe, type (amateur / professionnel)	Hérite de AbstractUser Django
Film	titre, annee_sortie, affiche, realisateur	FK → Catégorie · M2M → Acteur (via Casting)
Acteur	nom, prenom, age, photo	M2M → Film (via Casting)
Categorie	nom, descriptif	1-N → Film
Casting	—	Table intermédiaire Film ↔ Acteur
Commentaire	note (1–10), texte, date	FK → Film · FK → Personne

FONCTIONNALITÉS PRINCIPALES

Authentification

- Inscription (profil amateur / professionnel)
- Connexion / déconnexion
- Pages protégées par @login_required

Films

- Liste avec recherche (titre, réalisateur)
- Filtre par catégorie
- Fiche détail avec casting
- Ajout / modification / suppression
- Import CSV en masse

Commentaires & Notes

- Note de 1 à 10 par utilisateur
- Un seul avis par film par personne
- Moyenne globale, pro et amateur
- Meilleur / pire commentaire

Acteurs

- Liste avec recherche
- Fiche avec filmographie
- Ajout / modification / suppression
- Upload de photo

Catégories

- Liste et description
- CRUD complet
- Liaison automatique à l'import CSV

Administration

- Gestion des utilisateurs
- Interface d'administration Django (/admin)
- Modification du profil utilisateur

CYCLE DE TRAITEMENT D'UNE REQUÊTE

Le client envoie une requête HTTP → **Nginx** la reçoit sur le port 80 et la transmet à **Gunicorn** (port 8000 local) → Gunicorn invoque **Django** qui résout l'URL, appelle la vue correspondante, interroge **MariaDB** (VM 2) via l'ORM Django, génère une réponse HTML à partir des templates, et renvoie le tout au client. Les fichiers statiques (CSS, JS) et médias (images) sont servis directement par Nginx sans passer par Django.

4. PROCÉDURE DE DÉPLOIEMENT

PHASE 1 — VM 2 : CONFIGURATION DU SERVEUR DE BASE DE DONNÉES

1 Installer MariaDB

```
apt update
apt install mariadb-server
```

2 Autoriser les connexions distantes

```
nano /etc/mysql/mariadb.conf.d/50-server.cnf
```

Modifier la ligne `bind-address = 127.0.0.1` → `bind-address = 0.0.0.0`

```
systemctl restart mariadb
```

3 Créer la base, l'utilisateur et importer les données

```
mysql -u root
```

```
CREATE DATABASE cinebase_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'cinebase_user'@[IP_VM_DJANGO] IDENTIFIED BY 'motdepasseSAE';
GRANT ALL PRIVILEGES ON cinebase_db.* TO 'cinebase_user'@[IP_VM_DJANGO];
FLUSH PRIVILEGES;
EXIT;
```

Remplacer `[IP_VM_DJANGO]` par l'adresse IP réelle de la VM 1

```
mysql -u root cinebase_db < sauvegarde_cinebase.sql
```

PHASE 2 — VM 1 : CONFIGURATION DU SERVEUR WEB

1 Installer les dépendances système

```
apt update
apt install python3-venv python3-pip nginx default-libmysqlclient-dev pkg-config build-essential git
```

2 Cloner le dépôt GitHub (branche Alessandro)

```
cd /home/toto/
git clone -b Alessandro https://github.com/zeatyse/SAE-203-Albert.git
cd SAE-203-Albert/banque_films
```

3 Configurer les adresses IP dans settings.py

```
nano banque_films/settings.py
```

- `ALLOWED_HOSTS = ['[IP_VM_DJANGO]', 'localhost']`
- Dans le bloc `DATABASES` → `'HOST': '[IP_VM_SQL]'`

4 Créer l'environnement Python et lancer Gunicorn

```
python3 -m venv env
source env/bin/activate
pip install django gunicorn mysqlclient pillow
gunicorn banque_films.wsgi:application --bind 127.0.0.1:8000 --daemon
```

5 Configurer Nginx comme reverse proxy

```
nano /etc/nginx/sites-available/cinebase
```

```
server {
    listen 80;
    server_name [IP_VM_DJANGO];

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
    location /static/ {
        alias /home/toto/SAE-203-Albert/banque_films/static;
    }
    location /media/ {
        alias /home/toto/SAE-203-Albert/banque_films/media;
    }
}
```

6 Activer le site et redémarrer Nginx

```
rm /etc/nginx/sites-enabled/default
ln -s /etc/nginx/sites-available/cinebase /etc/nginx/sites-enabled/
systemctl restart nginx
```

L'application est accessible via `http://[IP_VM_DJANGO]` depuis n'importe quel navigateur du réseau.

PHASE 3 — GESTION DES MÉDIAS ET PLAN DE REPRISE D'ACTIVITÉ (PRA)

1 Configurer les permissions Nginx (dossier media)

Par défaut, Linux empêche Nginx de lire les fichiers dans le répertoire utilisateur. Il faut ouvrir les droits d'exécution sur le dossier parent :

```
chmod o+x /home/toto
chmod -R 755 /home/toto/SAE-203-Albert/banque_films/media
```

2 Générer les médias (scripts de peuplement)

Pour recréer les fichiers physiques (affiches et photos d'acteurs) directement depuis internet vers la VM 1 :

```
cd /home/toto/SAE-203-Albert/banque_films
source env/bin/activate
python vrais_films.py
python vrais_acteurs.py
```

Redémarrer Gunicorn après exécution : `kill gunicorn` puis relancer la commande `daemon`.

3 Plan de Reprise d'Activité (PRA) — Sauvegarde sur GitHub

Pour garantir la sécurité du projet, une sauvegarde complète de la base de données est poussée sur le dépôt Git distant.

Sur la VM 2 (SQL) — Exportation de la base :

```
mysqldump -u root cinebase_db > sauvegarde_finale.sql
scp sauvegarde_finale.sql toto@[IP_VM_DJANGO]:/home/toto/SAE-203-Albert/
```

Sur la VM 1 (Web) — Synchronisation sur GitHub :

```
cd /home/toto/SAE-203-Albert
git add sauvegarde_finale.sql
git commit -m "Sauvegarde de sécurité SQL"
git push -u origin Alessandro
```

Cette procédure garantit qu'en cas de panne de la VM 2, la base de données peut être restaurée depuis GitHub sur une nouvelle machine.

VÉRIFICATION DU BON FONCTIONNEMENT

Test	Commande / URL	Résultat attendu
Accès au site	http://[IP_VM_DJANGO]/	Page d'accueil CinéBase affichée
Statut Nginx	systemctl status nginx	active (running)
Statut MariaDB	systemctl status mariadb	active (running)
Connexion BDD distante	mysql -u cinebase_user -p -h [IP_VM_SQL] cinebase_db	Connexion SQL acceptée
Interface admin	http://[IP_VM_DJANGO]/admin/	Page de connexion Django admin